

OOP Problem of the Week — TOPIC: INTERFACE

Problem Statement: Library User Registration System

1. What is the Problem?

A library wants to build an **online registration and book-borrowing system**. The system must handle two types of users — **Children** and **Adults**. Each user type has different rules for registration and for borrowing books.

Since more user types may be added in the future (e.g. Teen users, Senior users), the system must be built in a **flexible and scalable** way using the concept of **Interfaces** in OOP.

2. Who Are the Users?

The system recognises users based on their **age**. The table below summarises the two user types:

User Type	Age Condition	Book Category Allowed	Return Period
KidUser	age < 12 years	Kids	10 Days
AdultUser	age > 12 years	Fiction	7 Days

3. What Should Each User Be Able to Do?

Every user in the system must support **two actions**:

Action / Method	What It Does
registerAccount()	Checks the user's age and registers them under the correct account type (Kid or Adult). Shows an error if age is wrong.
requestBook()	Checks whether the requested book category is allowed for that user. If yes, issues the book with the return deadline. Otherwise, shows an error.

4. Detailed Rules for Each User Type

4.1 KidUser — Rules

registerAccount():

- If age < 12 → Print: "You have successfully registered under a Kids Account"
- If age >= 12 → Print: "Sorry, Age must be less than 12 to register as a kid"

requestBook():

- If bookType is "**Kids**" → Print: "Book Issued successfully, please return the book within 10 days"
- If bookType is anything else → Print: "Oops, you are allowed to take only kids books"

4.2 AdultUser — Rules

registerAccount():

- If age > 12 → Print: "You have successfully registered under an Adult Account"
- If age <= 12 → Print: "Sorry, Age must be greater than 12 to register as an adult"

requestBook():

- If bookType is "**Fiction**" → Print: "Book Issued successfully, please return the book within 7 days"
- If bookType is anything else → Print: "Oops, you are allowed to take only adult Fiction books"

5. Test Cases (Given in the Problem)

Test Case 1 — KidUser

Object	age	bookType	Expected Output
KidUser	10	Kids	Registered as Kid ✓ 10-day borrow ✓
KidUser	18	Fiction	Age error ✗ Book type error ✗

Test Case 2 — AdultUser

Object	age	bookType	Expected Output
AdultUser	5	Kids	Age error ✗ Book type error ✗
AdultUser	23	Fiction	Registered as Adult ✓ 7-day borrow ✓

6. Why Use an Interface Here?

The problem says: "In future, more users/roles might be added." This is the exact situation where an **Interface** is the right choice. Here's why:

Reason	Explanation
Common contract	All user types must have registerAccount() and requestBook(). The interface guarantees this.
Flexibility	A new class like TeenUser can be added without changing KidUser or AdultUser at all.
Consistency	Every user class is forced to implement both methods — no method can be accidentally skipped.

Polymorphism	You can store any user type (Kid/Adult/Teen) in a LibraryUser variable and call the same methods on all of them.
--------------	--

7. Summary

- The library needs a registration system for **two types of users**: Kids and Adults.
- A user is a **Kid** if age < 12 — they can borrow **Kids** books for **10 days**.
- A user is an **Adult** if age > 12 — they can borrow **Fiction** books for **7 days**.
- Both user types must support **registerAccount()** and **requestBook()** methods.
- An **Interface (LibraryUser)** is created so that future user types can be added easily without breaking existing code.
- **KidUser** and **AdultUser** classes **implement** this interface with their own logic.